JOHANNES KEPLER
UNIVERSITY LINZ

Volodymyr Yelisieiev
12340334

April 30, 2026

NDSwin-JAX: Implementing and Evaluating an N-Dimensional Shifted Window Transformer in JAX

Technical Report
Practical Work in AI (BSc)

Institute for Machine Learning, Johannes Kepler University Linz

Public repository:
https://github.com/volodymyr-yelisieiev/ndswin-jax

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

Abstract

This report presents *NDSwin-JAX*, a JAX/Flax implementation of the Swin Transformer that was refactored into a genuinely N-dimensional framework. The core engineering objective was to preserve the shifted-window design of the original Swin architecture while removing its hard dependency on 2D image grids. In practice, that meant rewriting patch embedding, window partitioning and reversal, cyclic shifting, relative position indexing, and hierarchical stage composition so that they operate on arbitrary spatial dimensionalities controlled only by configuration.

The resulting system was evaluated on the latest preserved benchmark artifacts from April 29, 2026: 2D classification on CIFAR-10 and 3D voxel classification on a ModelNet40 export. The project also includes a CLI-driven workflow for training, random-search sweeps, queue execution, checkpointing, and report artifact extraction, so the evaluation covers both model behavior and the reproducibility of the surrounding experimentation pipeline. The saved artifacts show that the same codebase can complete a 48-trial CIFAR-10 sweep, a 15-trial ModelNet40 sweep, and full retraining of the best configuration for each dataset. Within that evidence, the best sweep validation accuracies reach 0.7344 on CIFAR-10 and 0.8061 on ModelNet40, while the corresponding final retraining runs restore checkpoints with validation accuracies of 0.7480 and 0.7421.

These results are sufficient to support the central claim of the project: the implementation successfully generalizes shifted-window transformers beyond 2D and remains trainable on both image and voxel inputs under a shared software stack. At the same time, the report stays conservative about what the saved evidence does *not* show. The repository snapshot contains validation metrics only, not held-out test measurements. It also contains no repeated-seed experiments, no error bars, and no statistical significance tests. Consequently, the contribution is best understood as a traceable implementation and empirical validation study rather than a definitive benchmark claim against published Swin baselines.

1. Introduction

Vision transformers have changed how visual recognition models are designed, but early transformer architectures often scaled poorly with input resolution because self-attention was computed globally over all tokens. The original Swin Transformer addressed this issue by replacing global attention with a hierarchy of local windowed attention blocks and a shifted-window mechanism that lets neighboring windows exchange information without restoring full quadratic attention cost across the entire image [4]. This design made transformer-based models much more practical for mainstream computer vision tasks and established Swin as one of the most influential hierarchical transformer backbones.

The limitation that motivated this project is that most implementations and much of the surrounding tooling remain strongly biased toward 2D imagery. In many research and engineering settings, however, the data are not naturally 2D. Medical scans, voxelized CAD objects, and other spatial measurements often live in three dimensions. Video and spatio-temporal signals can require even higher-dimensional reasoning. Reusing the shifted-window idea in these settings should be possible in principle, but real codebases often encode 2D assumptions in patch extraction, window bookkeeping, relative position indexing, data loading, and training orchestration. As a result, extending a 2D Swin implementation into a general N-dimensional system is less a matter of changing a single attention layer than of making the entire modeling and experimentation stack dimension-aware.

The goal of this practical work was therefore to build an N-dimensional shifted-window transformer in JAX and to verify that the resulting implementation can be trained in a realistic, traceable workflow. The emphasis of the project is not on proposing a new transformer variant. Instead, the contribution is an implementation and systems contribution: one codebase, one configuration-driven command surface, and one training stack that can support both 2D image classification and 3D voxel classification by changing configuration rather than rewriting architecture-specific code. This project also aimed to make the surrounding workflow credible from a research-practice perspective by supporting hyperparameter sweeps, queue-based job execution, logging, checkpointing, and dataset validation before a run starts.

The repository snapshot analyzed in this report contains exactly the kind of evidence needed for such a claim. First, it includes an end-to-end 48-trial sweep on CIFAR-10, followed by full retraining of the selected configuration and restoration of the best validation checkpoint at 0.7480, while the sweep itself reached 0.7344. Second, it includes a 15-trial sweep on a voxelized ModelNet40 dataset, followed by full retraining of the selected configuration and restoration of the best validation checkpoint at 0.7421, while the sweep itself reached 0.8061. Third, it includes queue and resume logs showing how the workloads were orchestrated through the same CLI-based pipeline, including the ModelNet40 rerun after a train-derived validation split was created. These artifacts demonstrate that the project is not merely a collection of isolated modules, but a functioning experimentation framework.

This report is structured as follows. section 2 situates the implementation in the context of the original Swin paper, the JAX/Flax ecosystem, and the datasets used here. section 3 explains how the architecture and training system were generalized from fixed 2D assumptions to arbitrary spatial dimensionalities. section 4 documents the experimental setup, the datasets, the hardware, and the sweep procedures. section 5 interprets the validation results, including the gap between sweep-best and final retraining performance in the latest queue run. Finally, section 6 summarizes the main constraints of the preserved

evidence, especially the lack of held-out test evaluation and repeated-seed uncertainty estimates. The report deliberately stays within those limits so that every conclusion can be traced back to the saved artifacts rather than to assumptions or reconstructed claims.

2. Related Work

NDSwin-JAX builds directly on the Swin Transformer of Liu et al. [4]. Swin made vision transformers more practical by replacing global attention with local window attention and shifted windows. This retained a hierarchical feature pyramid while reducing the cost of attention on dense visual grids.

The same idea has already proved useful beyond ordinary 2D classification. Video Swin adapts shifted-window attention to video recognition by adding a temporal dimension while preserving the locality and hierarchy of the original model [5]. Swin3D applies a Swin-style backbone to sparse 3D scene understanding, showing why local attention and relative position information are natural tools for large 3D inputs [8]. These projects motivate the central engineering goal of this work: implement the spatial operators once, over tuples of dimensions, instead of maintaining separate 2D and 3D code paths.

Medical imaging is especially relevant because many datasets are naturally volumetric. UNETR reformulates 3D medical segmentation as transformer-based sequence modeling with a U-shaped decoder [2]. Swin UNETR then combines a hierarchical Swin encoder with medical segmentation decoders and self-supervised pre-training on large CT collections [1, 6]. nnFormer is another volumetric transformer that interleaves convolution and attention for 3D segmentation [9]. Together, these works show that high-dimensional transformer backbones are not just a software curiosity; they are a practical direction for volumetric data.

This report is narrower than those papers. It does not propose a new medical segmentation benchmark or claim state-of-the-art accuracy. Its contribution is an auditable JAX/Flax implementation of the shifted-window backbone that can be configured for different spatial dimensionalities, plus the training and queue machinery needed to run 2D and 3D experiments reproducibly.

3. Methods and Implementation

The central implementation task was to remove implicit 2D assumptions from the Swin pipeline while preserving the original architectural pattern. In the analyzed repository snapshot, the resulting model is exposed as `NDSwinTransformer` and configured through `num_dims`, `patch_size`, and `window_size`. The model still follows the familiar Swin structure: patch embedding, multiple hierarchical transformer stages, and a classifier head. The difference is that spatial operators now work on tuples of arbitrary length instead of fixed height–width pairs.

At the model level, patch embedding converts channel-first tensors of shape $(B, C, *\text{spatial})$ into token grids whose dimensionality depends only on the configured patch size. The generalized window operators then partition tensors of shape $(B, *\text{spatial}, C)$ into local windows, reverse that partitioning after attention, and apply cyclic shifts along each spatial axis. Relative position bias indexing is likewise computed from the configured window shape. These are exactly the components that fail first in a Swin implementation that assumes only two spatial coordinates. Replacing those

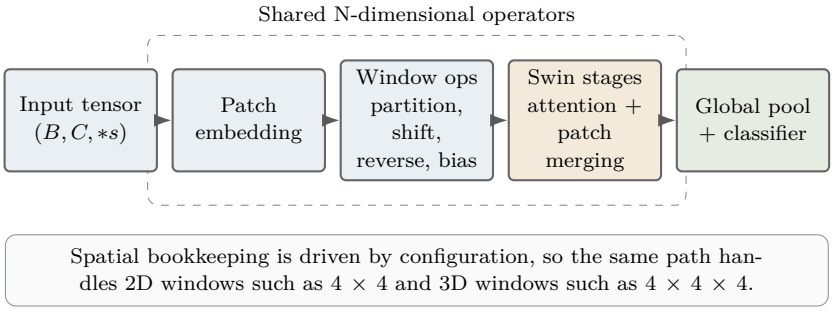

Figure 1: Overview of the N-dimensional shifted-window pipeline used in NDSwin-JAX. The grouped middle blocks are the generic spatial operators: they reuse the same code path for 2D image windows and 3D voxel windows by reading `num_dims`, `patch_size`, and `window_size` from configuration.

Table 1: Execution environment extracted from the preserved artifact set. The software versions are grounded in `pyproject.toml`; the runtime device selection is confirmed by the saved training logs.

Item	**Value**
CPU	Intel Xeon E5-2630 v4, 20 logical CPUs
GPU inventory	4 × NVIDIA GeForce GTX 1080 Ti, 11264 MiB each
RAM	125 GiB system memory
Runtime device selection	4 GPUs selected automatically for both preserved final training runs
Core software	JAX 0.7.1, jaxlib 0.7.1, Flax, Optax, Orbax checkpointing

assumptions with tuple-based shape arithmetic lets the same attention block process both 4×4 image windows and $4 \times 4 \times 4$ voxel windows.

The implementation also preserves the hierarchical structure of Swin. The model stacks multiple stages, each with its own depth, number of heads, and stochastic depth schedule. Between stages, patch merging reduces spatial resolution while increasing the embedding dimension. Because these transformations operate on generic spatial tuples, the same stage logic applies to both the CIFAR-10 model and the ModelNet40 model analyzed here. In practice, moving from 2D to 3D becomes a configuration change rather than an architectural rewrite.

The project extends beyond the model definition into the training system. The repository's canonical interface is the `ndswin` CLI, which supports training, sweeps, auto-sweeps, queue execution, dataset export, and validation. This matters for reproducibility because the same command surface launches both the 2D and 3D workflows. In the preserved artifact set, a queue file schedules one auto-sweep-plus-retrain workflow for CIFAR-10 and one for ModelNet40. The evidence therefore covers both the model and the orchestration layer expected in a practical research project.

Training itself is implemented with JAX-based optimization and data-parallel sharding. The trainer constructs a JAX mesh over the visible devices and shards batches along the batch dimension using `NamedSharding`. This is relevant because the kept April logs show that the preserved final runs selected four GPUs automatically rather than relying on a manually fixed device list. The code chooses only devices that satisfy memory and

utilization thresholds, so the report does not overstate the hardware actually used for the preserved experiments.

Finally, the implementation includes guard rails around data and configuration. Dataset contracts are validated before training starts, the ModelNet40 export is described by a manifest containing split counts and class information, and the sweep and queue layers record summaries to JSON. These details matter scientifically because they make the evidence auditable. The results reported later are extracted from structured artifacts that preserve configuration, runtime metadata, and metric histories rather than from informal notes or screenshots.

4. Experimental Setup

The experiments in this report are restricted to the latest preserved benchmark artifacts, completed on April 29, 2026. This is intentional: the goal is to document what was actually built and measured in the final benchmark pass, not to mix evidence across superseded runs. The CIFAR-10 workflow completed through the benchmark queue. The first ModelNet40 queue attempt stopped before training because the exported dataset did not yet contain a validation directory; after a deterministic validation split was created from the training split, the ModelNet40 auto-sweep was resumed and completed successfully. All quantitative results below therefore come from the April 29 sweep summaries, metric histories, best configurations, dataset manifest, and logs copied under `report/data/sources/`.

The first workload is CIFAR-10, a 10-class image classification task with 32×32 RGB inputs [3]. In the recorded final run, the split comprises 45,000 training samples, 5,000 validation samples, and 10,000 test samples. The saved experiment evaluates the validation split at the end of training, so the report treats the test split as unused supporting context rather than as a source of headline metrics. The selected retraining configuration uses `num_dims = 2`, a patch size of 2×2, a window size of 4×4, and an embedding dimension of 96.

The second workload is a voxelized ModelNet40 export [7]. The repository manifest describes 8,858 training samples, 985 train-derived validation samples, and 2,468 test samples, with one input channel and a spatial resolution of 32^3. This experiment uses `num_dims = 3`, which activates the generalized 3D path in patch embedding, window partitioning, shifting, and patch merging. The selected retraining configuration uses an embedding dimension of 72, showing that the stabilized 3D sweep did not select the smallest architecture variant.

Selected retraining configurations.

- **CIFAR-10:** patch size 2×2, window size 4×4, embed dimension 96, batch size 128, learning rate 0.0004300210, weight decay 0.0104108183, drop rate 0.05, drop path 0.3, warmup 20 epochs, mixup 0.0, cutmix 0.5, cutout 0, color jitter enabled.
- **ModelNet40:** patch size $2 \times 2 \times 2$, window size $4 \times 4 \times 4$, embed dimension 72, batch size 64, learning rate 0.0012622739, weight decay 0.0070498347, drop rate 0.0, drop path 0.2, warmup 3 epochs.

Hyperparameter search is performed through configuration-driven random search. The CIFAR-10 sweep evaluates 48 trials with validation accuracy as the selection metric and a 60-epoch budget per trial. All 48 trials completed successfully. The ModelNet40 sweep

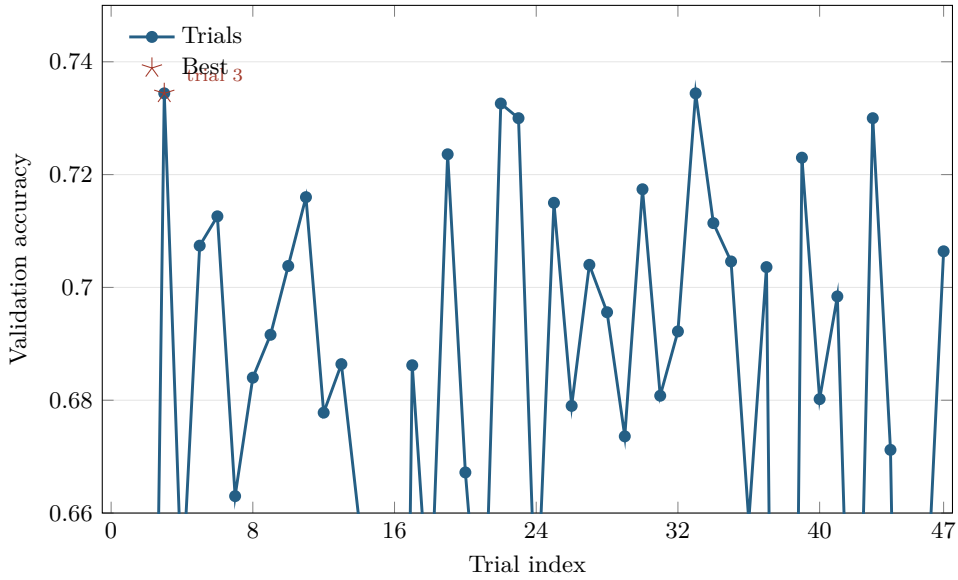

Figure 2: Validation accuracy across the 48 CIFAR-10 sweep trials in the kept April 29 queue run. The highlighted star marks the CLI-selected best recorded trial, trial 3, which reached 0.7344 under the 60-epoch sweep budget.

evaluates 15 trials with the same metric and a 10-epoch budget per trial, and all 15 trials completed successfully as well. This is important because it shows that the later stable pipeline no longer suffered from the invalid warmup schedules that had affected earlier exploratory 3D sweeps.

After sweep selection, each task is retrained with the best saved configuration. For CIFAR-10, the queue launches the final retraining with 600 requested epochs, but early stopping terminates the run at epoch 176 and restores the best validation checkpoint from epoch 136. For ModelNet40, the resumed workflow launches final retraining with 200 requested epochs, early stopping terminates the run at epoch 27, and the best validation checkpoint is restored from epoch 7. The early-stopped best state, not the nominal epoch budget, therefore determines the final preserved validation metric.

The experiments were executed through the repository's queue and tmux-backed workflow mechanisms, which are themselves part of the practical contribution. The queue log records a completed CIFAR-10 auto-sweep-plus-retrain job with a total elapsed time of 124,965.6 seconds, followed by the stopped ModelNet40 attempt that exposed the missing validation split. The resumed ModelNet40 auto-sweep then ran from 15:54 to 17:16 on April 29 and completed successfully. These workflow-level artifacts show both the intended unattended benchmark path and the recovery path used when dataset validation correctly blocked an unsafe run. The committed extraction inputs are listed in Appendix A.

5. Results and Interpretation

The most important result of this project is qualitative before it is quantitative: one codebase successfully trained on both a 2D image benchmark and a 3D voxel benchmark with no architecture rewrite between the two tasks. The same model family, training stack, and CLI workflow were reused across both workloads, with task-specific behavior

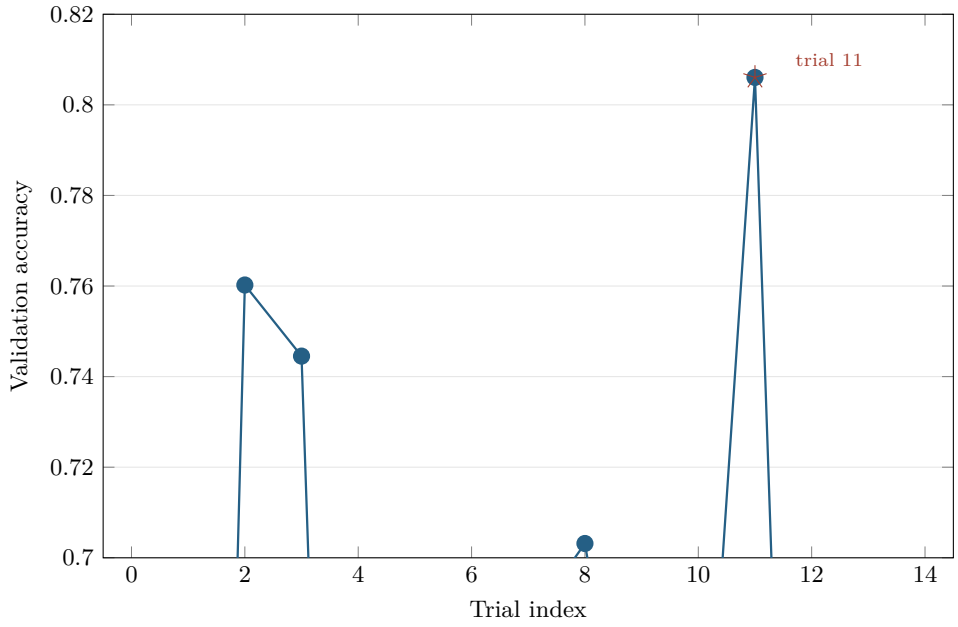

Figure 3: Validation accuracy across the 15 ModelNet40 sweep trials in the resumed April 29 run. The highlighted star marks trial 11, which reached 0.8061, and the absence of failed trials confirms that the stabilized sweep space removed the earlier invalid warmup combinations.

determined by configuration. That is the core empirical validation of the N-dimensional implementation.

For CIFAR-10, the 48-trial sweep completed without failures. The CLI-selected best sweep trial was trial 3, which achieved a validation accuracy of 0.7344 and a top-5 validation accuracy of 0.9672 under the 60-epoch sweep budget. After retraining the selected configuration with the longer schedule, the preserved best validation checkpoint reached a validation accuracy of 0.7480, a top-5 validation accuracy of 0.9762, and a validation loss of 0.8020. The run used 48,809,188 parameters and took 6,676.51 seconds end to end, with the best checkpoint found at epoch 136 before early stopping triggered at epoch 176. The training curves in Figure 4 show that the long retraining remained stable and modestly improved on the sweep-best validation score.

For ModelNet40, the stabilized sweep is informative precisely because it no longer exposes invalid configuration failures. All 15 trials completed successfully, and the best sweep configuration, trial 11, achieved a validation accuracy of 0.8061 and a top-5 validation accuracy of 0.9499. Full retraining of that selected configuration did *not* improve on the sweep-best score; instead, the preserved best validation checkpoint reached 0.7421, with a top-5 validation accuracy of 0.9306 and a validation loss of 0.9640. The final model used 16,423,126 parameters and completed in 990.99 seconds, with the best checkpoint found at epoch 7 and early stopping at epoch 27. This does *not* imply that the architecture fails on 3D data. Rather, it shows that under this particular resumed run, the chosen long retraining schedule did not translate the sweep-best configuration into a stronger final checkpoint.

The comparison between sweep-best and final-retrain performance is summarized in Table 2. In this latest artifact set, CIFAR-10 improves after full retraining, while ModelNet40 remains below its best short-budget sweep trial. That mixed outcome is informative. It shows that the orchestration pipeline is stable enough to complete both stages

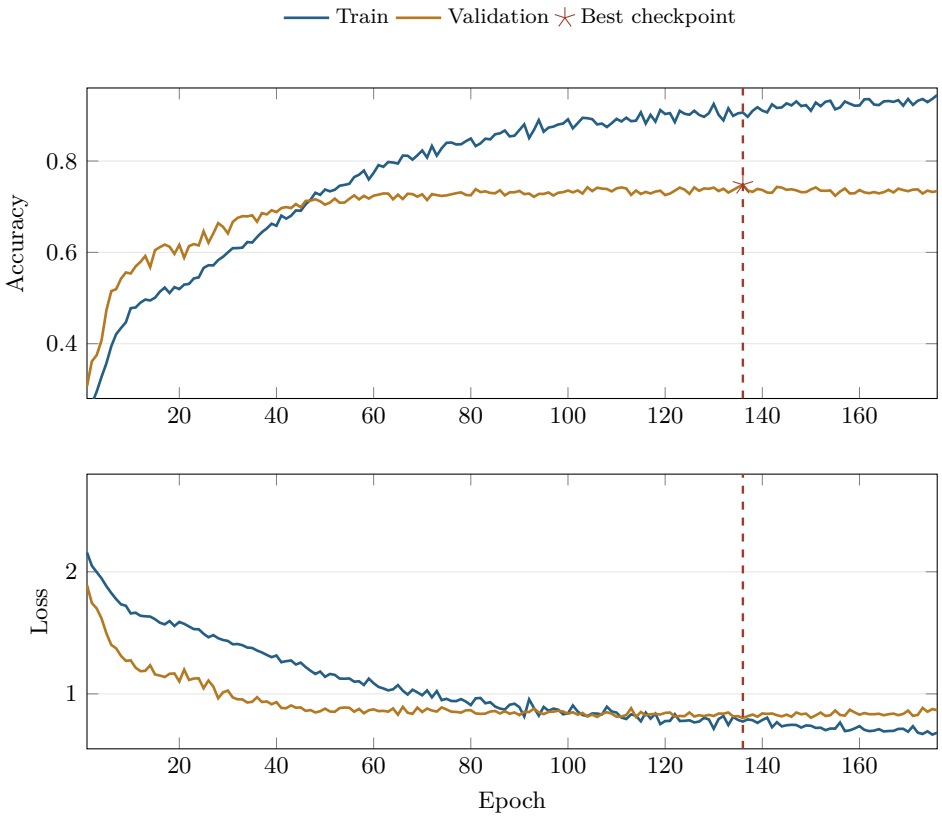

Figure 4: Training history for the final CIFAR-10 retraining run. The dashed marker highlights the best restored checkpoint at epoch 136 with validation accuracy 0.7480; early stopping terminated the run at epoch 176.

with traceable, repeatable artifact generation, but it also shows that a long retraining schedule is not automatically better than the best sweep checkpoint under the chosen configuration and stopping rules. The distinction between "best sweep trial" and "best final retrain" therefore needs to remain explicit. No error bars are shown because the preserved evidence for this submission consists of single retained runs rather than repeated-seed experiments.

From a software engineering perspective, the workflow results are also meaningful. The CIFAR job completed in 124,965.6 seconds inside the benchmark queue, and the resumed ModelNet40 auto-sweep completed successfully in a separate tmux-backed run after the validation split was repaired. The saved logs also show that both final runs used four automatically selected GPUs rather than a manually fixed device list, which demonstrates basic resource awareness in the training pipeline.

Overall, the results support a restrained but clear interpretation. NDSwin-JAX is not yet supported by the kind of repeated, test-set benchmark campaign that would justify strong state-of-the-art claims. Nevertheless, the preserved latest-run artifact set shows that the implementation is functional, configurable, and reusable across dimensionalities. The project succeeds as a practical work in applied machine learning engineering: it translates a known architecture into a broader implementation setting and validates that translation with credible, traceable experiments, while also revealing where retraining settings still need improvement.

Figure 5: Training history for the final ModelNet40 retraining run. The dashed marker highlights the best restored checkpoint at epoch 7 with validation accuracy 0.7421; early stopping stopped the run at epoch 27.

6. Limitations and Conclusion

Several limitations of the preserved evidence need to be stated explicitly. The most important is that all headline metrics in this report are validation metrics, not held-out test results. The repository contains test splits for both datasets, but the saved final evaluation step in the analyzed runs uses the validation loader. The report can therefore claim successful validation performance, but it cannot claim final generalization performance in the stricter benchmark sense. For the same reason, these numbers should not be compared casually to published Swin results reported on held-out evaluation splits under standardized protocols.

The second major limitation is the absence of repeated-seed experiments, error bars, and statistical significance tests. The project supports reproducibility in the engineering sense through configuration files, saved metrics, queue logs, and deterministic artifact paths. However, reproducibility in the stronger statistical sense would require rerunning the same configuration multiple times and quantifying variation across seeds or training stochasticity. Those measurements are not available in the preserved artifact set, so the report avoids any language suggesting statistical confidence beyond the single recorded runs.

The third limitation concerns the gap between sweep-best and final-retrain performance. In the kept April 29 artifact set, CIFAR-10 improved after retraining but ModelNet40 restored a weaker checkpoint than the best short-budget sweep trial. That does not under-

Table 2: Summary of the preserved validation results. Sweep metrics correspond to the best completed sweep trial; final metrics correspond to the best restored checkpoint after full retraining.

Dataset	Sweep	Final	Top-5	Parameters	Time (s)	Best Ep.
CIFAR-10	0.73	0.75	0.98	48,809,188	6,676.51	136
ModelNet40	0.81	0.74	0.93	16,423,126	990.99	7

mine the implementation claim, but it does mean the final-stage training policy should not be treated as automatically optimal. Better retraining policies, stronger schedule transfer from sweep to full run, or repeated validation of the selected best configuration would strengthen the empirical story.

Another limitation is the scope of the empirical comparison itself. The report evaluates only two workloads: CIFAR-10 as a 2D benchmark and a voxelized ModelNet40 export as a 3D benchmark. This is enough to demonstrate dimensional generality, but not to claim broad superiority across datasets, tasks, or modalities. The ModelNet40 experiment also depends on a voxelized export pipeline rather than on the raw representation used in some other 3D learning pipelines. Its validation split was created from the training split after dataset validation correctly rejected the original export, so the present report treats the 3D result as evidence that the implementation works on volumetric data, not as a universal statement about 3D vision performance.

With those boundaries in place, the overall conclusion is still positive. The project successfully implemented a reusable N-dimensional shifted-window transformer framework in JAX and demonstrated that it can be trained in a practical workflow on both 2D images and 3D voxels. The architecture-level generalization is real in code, not merely conceptual: patch embedding, window partitioning, shifting, relative position handling, and stage composition all operate on configurable spatial dimensionality. The surrounding experimentation framework is likewise functional, supporting sweeps, queue execution, checkpointing, and structured metric export.

In that sense, the work fulfills the core objective of a practical AI project based on the original Swin paper. It translates a strong published idea into a broader implementation setting, exposes the engineering decisions needed to make that translation work, and produces valid empirical evidence that the resulting system can learn on two different data modalities. Future work should prioritize held-out test evaluation, repeated-seed uncertainty estimates, stronger retraining policies after sweep selection, and direct comparison to published Swin baselines. Those additions would strengthen the scientific completeness of the evidence, but they are not required to support the central conclusion of this report: NDSwin-JAX is a credible, traceable N-dimensional implementation of shifted-window transformers in JAX.

References

[1] Ali Hatamizadeh, Vishwesh Nath, Yucheng Tang, Dong Yang, Holger Roth, and Daguang Xu. 2022. Swin UNETR: Swin Transformers for Semantic Segmentation of Brain Tumors in MRI Images. *arXiv preprint arXiv:2201.01266*.

[2] Ali Hatamizadeh, Yucheng Tang, Vishwesh Nath, Dong Yang, Andriy Myronenko, Bennett Landman, Holger Roth, and Daguang Xu. 2022. UNETR: Transformers for 3D Medical Image Segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 1748–1758.

[3] Alex Krizhevsky. 2009. Learning Multiple Layers of Features from Tiny Images. Technical report. University of Toronto.

[4] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022. DOI: 10.1109/ICCV48922.2021.00986.

[5] Ze Liu, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu. 2022. Video Swin Transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3202–3211.

[6] Yucheng Tang, Dong Yang, Wenqi Li, Holger Roth, Bennett Landman, Daguang Xu, Vishwesh Nath, and Ali Hatamizadeh. 2022. Self-Supervised Pre-Training of Swin Transformers for 3D Medical Image Analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20730–20740.

[7] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1912–1920. DOI: 10.1109/CVPR.2015.7298801.

[8] Yu-Qi Yang, Yu-Xiao Guo, Jian-Yu Xiong, Yang Liu, Hao Pan, Peng-Shuai Wang, Xin Tong, and Baining Guo. 2023. Swin3D: A Pretrained Transformer Backbone for 3D Indoor Scene Understanding. *arXiv preprint arXiv:2304.06906*.

[9] Hong-Yu Zhou, Jiansen Guo, Yinghao Zhang, Lequan Yu, Liansheng Wang, and Yizhou Yu. 2021. nnFormer: Interleaved Transformer for Volumetric Segmentation. *arXiv preprint arXiv:2109.03201*.

Appendix A. Committed Artifact Copies

The public code for this project is available at https://github.com/volodymyr-yelisieiev/ndswin-jax. The quantitative evidence in this report is tied to committed experiment-artifact copies under `report/data/sources/`. The derived plot tables in `report/data/*.csv` are regenerated from those local copies, or from a newer benchmark artifact set, by `report/extract_report_data.py`. The restored-best checkpoints were also packaged as Hugging Face model repositories so the final weights can be inspected independently of the local training machine.

The figures and tables in this report are derived from the following committed artifact copies, relative to `report/data/sources/`:

Sweep summaries	`outputs/sweeps/cifar10_tuned_hyperparam_sweep/summary.json`
	`outputs/sweeps/modelnet40_stable_hyperparam_sweep/summary.json`
Final metrics	`outputs/cifar10/cifar10_tuned_8021c956_20260429_064522/checkpoints/metrics.json`
	`outputs/volume_folder/modelnet40_f67c9398_20260429_165927/checkpoints/metrics.json`
Logs	`logs/queue_20260427_214146.json`
	`logs/tmux/benchmark/benchmark_20260427_214145.log`
	`logs/tmux/auto_sweep/autosweep_20260429_155416.log`
	`logs/cifar10/cifar10_tuned_8021c956_20260429_064522.log`
	`logs/volume_folder/modelnet40_f67c9398_20260429_165927.log`
Dataset manifest	`data/modelnet40/dataset_manifest.json`
Best configs	`configs/auto_best/best_cifar10_cifar10_tuned_05f7e1b8_20260427_234823_trial003.json`
	`configs/auto_best/best_volume_folder_modelnet40_e548c891_20260429_164145_trial011.json`
Published weights	https://huggingface.co/volodymyr-yelisieiev/ndswin-cifar10-final
	https://huggingface.co/volodymyr-yelisieiev/ndswin-modelnet40-final
Local weight exports	`outputs/hf_exports/cifar10_final/`
	`outputs/hf_exports/modelnet40_final/`

Code-level implementation claims are grounded in the tracked repository files `pyproject.toml`, `src/ndswin/training/trainer.py`, and `src/ndswin/utils/gpu.py`.